

Project LEE

Learning and Evolving Environment

Persistent Founder Operating Environment and Lamont Labs Orchestration Layer

1. Core Definition

Project LEE — Learning and Evolving Environment — is the persistent operating intelligence layer for Lamont Labs.

LEE is designed to understand the full operational picture across projects, people, systems, services, objectives, decisions, events, relationships, dependencies, risks, costs, and accumulated experience.

LEE is not intended to perform every specialized task herself.

Instead:

«LEE understands, coordinates, supervises, remembers, prioritizes, and orchestrates. Specialized systems perform their own specialist work. APIs connect them.»

LEE therefore sits above the broader Lamont Labs ecosystem.

Her job is to continuously answer questions such as:

- What is happening?
- What changed?
- What deserves attention?
- What am I waiting on?
- What is blocked?
- Which project is moving?
- Which project is stagnating?
- Which systems are healthy?
- What can continue without intervention?
- What requires approval?
- What system should handle this?
- What AI reasoning is actually required?
- What did this cost?
- What happened as a result?
- What have we learned from similar outcomes?

2. Fundamental Architecture

The long-term architecture is built around several strict separations.

LEE

Owns:

- operational context
- persistent memory
- organizational understanding
- project intelligence
- people and relationships
- priorities
- objectives
- orchestration
- workflow coordination
- system supervision
- portfolio intelligence
- historical continuity
- learning from outcomes

CIL

Owns:

- cognitive reuse
- reasoning resolution
- AI cost minimization
- model selection
- model-class selection
- cognitive routing
- T1/T2/T3 logic
- cognitive asset logic
- contradiction handling
- drift handling
- routing economics

CerbaSeal

Owns:

- consequential-action governance
- policy evaluation
- authorization
- ALLOW / HOLD / REJECT decisions
- approval-chain enforcement
- policy-pack interpretation
- governance evidence

Replit AI Bridge

Owns:

- access to Replit-managed AI integrations
- execution of CIL-selected Replit model requests
- returning model results and usage metadata

Connected Lamont Labs Systems

Own their own specialist domains.

Examples may include:

- Greyline
- QuantraCore
- future Lamont Labs systems

The permanent rule is:

«LEE may contain API clients and system contracts for CIL, CerbaSeal, and other systems, but not duplicate implementations of their specialist logic.»

3. CIL and CerbaSeal Are Not Part of LEE

This is an important architectural correction.

CIL and CerbaSeal should not exist as coded internal versions inside Project LEE.

They are separate Lamont Labs systems.

LEE communicates with them strictly through authenticated APIs.

LEE may contain:

- API client
- endpoint configuration
- authentication logic
- request schema
- response schema
- health checks
- timeout handling
- retry handling
- service-state tracking
- correlation IDs
- normalized events
- UI representation

LEE must not contain a second implementation of:

CIL logic

- T1 logic
- T2 logic
- T3 logic
- model-selection rules
- cost optimization
- cognitive asset selection
- contradiction arbitration
- cognitive crystallization
- CIL-specific confidence logic

CerbaSeal logic

- policy-pack evaluation
- ALLOW/HOLD/REJECT decision logic
- approval-chain evaluation
- governance invariants
- decision-envelope creation
- authorization-expiry rules
- evidence-bundle generation

There must be one authoritative implementation of each.

4. Why This Separation Matters

Duplicating CIL or CerbaSeal logic inside LEE would eventually create competing authorities.

For example:

CIL could evolve its routing policy.

If LEE contains an older copy of CIL logic, the systems could disagree about:

- whether reuse is safe
- which model to use
- whether escalation is required
- what the cheapest valid route is

Likewise, CerbaSeal could update policy while LEE still contains older embedded governance logic.

That could create a much more serious conflict.

Therefore:

«CIL is authoritative for cognitive routing.»

«CerbaSeal is authoritative for consequential-action governance.»

«LEE is authoritative for operational orchestration and accumulated context.»

5. One Lee, One Brain, Multiple Surfaces

LEE should remain one persistent intelligence environment.

The architectural principle is:

«One Lee. One authoritative Brain. Multiple surfaces.»

The major surfaces are:

LEE Desktop

The full operating environment, eventually running on the K6.

LEE Mobile Companion

The Android interface for portable access, capture, alerts, approvals, Ask Lee, and lightweight system control.

Replit Infrastructure

A cloud component that preserves access to Replit-managed AI integrations and other selected cloud capabilities.

None of these becomes a separate authoritative Lee.

6. The LEE Brain

The LEE Brain is the durable asset.

It contains accumulated operational intelligence such as:

- Identity
- Constitution
- Event history
- Facts
- Interpretations
- Assumptions
- Strategic Anchors
- Decision Heuristics
- Institutional Knowledge
- Organizational Memory
- Operational Memory
- project state
- people
- relationships
- objectives
- dependencies
- portfolio structure
- project momentum
- execution readiness
- opportunities
- simulations
- reviews
- experiences
- lessons
- provenance
- Why Chains

- confidence
- uncertainty

Models can change.

Connected systems can change.

The K6 can change.

Replit can change.

The Brain should survive all of them.

7. Portability

LEE must remain portable from the beginning.

The Brain should be:

- exportable
- versioned
- restorable
- migratable
- auditable
- independently verifiable

The K6 is intended to become LEE's primary local home.

It should not become a permanent architectural dependency.

The principle is:

«Move Lee, not rebuild Lee.»

8. LEE Core

LEE Core contains the internal systems that truly belong to LEE.

Major components include:

- Identity Engine
- Constitution Engine
- Intent Engine
- Understanding Pipeline
- Event Log
- Domain Events
- Query Engine
- Context Economy
- Fact Ledger
- Interpretation Ledger
- Assumption Ledger
- Strategic Anchors
- Decision Memory
- Organizational Memory
- Intelligence Graph
- Decision Impact Graph
- Digital Twin Timeline
- Time Machine
- Strategy Engine
- Reflection Engine
- Explanation Engine
- Simulation Engine
- Operational Intelligence Engine
- Executive Loop
- Operational Confidence
- Project Momentum
- Opportunity Engine
- Portfolio Intelligence
- Portfolio Dependency Graph
- Execution Readiness
- Resource Allocation
- Operational Reviews
- Institutional Knowledge
- Operational Self-Improvement
- System Economics
- System Registry
- Capability Registry
- Universal Systems API Layer
- Model Router
- backup and recovery
- System Manifest
- Self-Test Framework

CIL and CerbaSeal are intentionally absent from this list as internal implementations.

9. Identity

Identity defines who Lee is and how she should operate.

It should be:

- structured
- persistent
- versioned
- owner-controlled

Identity includes areas such as:

- who Lee is
- why Lee exists
- responsibilities
- what Lee must protect
- what Lee must never do
- priority order
- success definition
- interruption threshold
- silence threshold
- escalation threshold
- ask threshold
- observation threshold

Identity should be consulted before normal reasoning or action.

10. Constitution

The Constitution defines non-negotiable operating boundaries.

Identity answers:

«Who am I?»

The Constitution answers:

«What am I allowed to do?»

The core LEE request sequence remains:

Identity

→ Constitution

→ Intent

→ Context

After that, LEE may call external systems if necessary.

11. Event-Sourced History

LEE should maintain an append-only canonical history of important events.

Examples:

- project created
- source captured
- Fact created
- Interpretation created
- assumption invalidated
- objective updated
- system connected
- capability registered
- workflow started
- workflow completed
- CIL routing decision received
- model request executed
- CerbaSeal decision received
- owner approval received
- connected system degraded
- backup created
- restore verified
- review completed
- Institutional Knowledge promoted

Current state should increasingly be reconstructable from events.

This supports:

- recovery
- historical analysis

- accountability
- Time Machine
- learning from outcomes
- auditing

12. Facts and Interpretations

LEE must preserve a permanent structural distinction between evidence and conclusions.

Fact

A sourced claim about reality.

Interpretation

A conclusion derived from evidence.

Example:

Fact

Greyline completed seven production jobs.

Interpretation

Greyline's current workflow appears operationally stable.

Interpretations should retain:

- evidence references
- confidence
- provenance
- Why Chain
- timestamp
- creator

13. Provenance

LEE should always be able to answer:

«Why do you believe this?»

Important conclusions should remain traceable through:

Recommendation

- reasoning
- Interpretation
- Facts
- source evidence

This should apply to:

- strategy
- priorities
- opportunities
- simulations
- readiness
- relationship assessments
- operational conclusions
- Institutional Knowledge

14. Assumptions

Unverified beliefs should be explicitly tracked as Assumptions.

Possible lifecycle:

- active
- under review
- invalidated
- archived

When an assumption becomes invalid, LEE should know what conclusions depended on it.

15. Strategic Anchors

Strategic Anchors preserve durable owner-defined commitments.

Examples may include:

- architectural commitments
- foundational product philosophy
- rejected approaches
- long-term priorities

Anchors should not silently age away.

Contradictions should be surfaced.

16. Decision Memory

LEE should infer recurring decision heuristics from observed decisions.

Examples:

- portability is preferred over vendor lock-in
- systems should remain independently useful
- proof is preferred over architectural claims

Decision Heuristics should include:

- evidence
- confidence
- exceptions
- first observed
- last reinforced

17. Institutional Knowledge

LEE should learn durable organizational knowledge through repeated reality.

The progression remains:

Event

- Experience
- Lesson
- Pattern
- Institutional Knowledge

Institutional Knowledge should not be created from one isolated observation.

It should be earned through repeated supported outcomes.

18. Query Engine

LEE's intelligence engines should retrieve operational knowledge through one universal Query Engine.

It should handle:

- authorization
- structured search
- semantic search
- Fact/Interpretation separation
- freshness
- aging
- confidence
- trust
- project scope
- people scope
- timeline scope
- ranking
- Context Economy preparation

This prevents individual engines from developing incompatible views of reality.

19. Context Economy

LEE should not send its entire Brain to AI models.

Context should be selected based on factors such as:

- goal match
- recency
- importance
- relationship relevance
- project activity
- confidence
- trust
- current operating mode

- current objective

The goal is:

«Use the smallest context packet capable of producing a reliable result.»

20. Operational Intelligence

Operational Intelligence answers:

«What deserves attention right now?»

LEE should continuously identify:

- Active Priority
- What Changed
- What Is Drifting
- What Is Waiting
- What Is Blocked
- What Is At Risk
- What Can Wait
- What Should Be Ignored Today

This should be evidence-driven.

21. Executive Loop

LEE's operational heartbeat remains:

Observe

- Understand
- Prioritize
- Decide
- Prepare
- Wait
- Review
- Repeat

The Executive Loop should:

- persist through restart
- react to important events
- monitor connected systems
- update priorities
- prepare useful actions
- wait when nothing needs intervention
- review outcomes
- feed experience back into learning

22. Portfolio Intelligence

LEE understands Lamont Labs as one portfolio.

She should understand:

- shared infrastructure
- system dependencies
- project momentum
- readiness
- resource competition
- common technologies
- shared customers
- opportunities
- risks
- potential reuse

This enables questions such as:

- Which project deserves attention?
- Which project is closest to commercial readiness?
- Which systems depend on this service?
- Where is effort being duplicated?
- What system failure would have the largest blast radius?

23. LEE Desktop on the K6

The K6 should eventually become LEE's primary local runtime.

The K6 can host:

- LEE Core
- database
- Event Log
- Semantic Index
- Query Engine
- Executive Loop
- Operational Intelligence
- System Registry
- Capability Registry
- local background workers
- safe local models where appropriate
- backup services
- desktop notifications

LEE should ultimately behave like a proper desktop application.

The user experience should be:

«Open Lee.»

24. Desktop Interface

The desktop application should become the complete operating control center.

Major areas include:

Today

Current priorities and changes.

Ask Lee

Full interaction with the Brain.

Projects

Individual project state.

Portfolio

Cross-project view.

Systems

Connected projects and services.

People

Relationships and waiting loops.

Objectives

Cross-project goals.

Knowledge

Facts, Interpretations, Assumptions, Anchors, Decision Memory, Institutional Knowledge.

Timeline

Operational history.

Opportunities

Detected cross-system leverage.

Simulations

What-if scenarios.

Reviews

Weekly, monthly, quarterly, annual.

Governance

CerbaSeal-backed consequential-action status.

Economics

LEE and connected-system cost.

Health

LEE and system health.

Backups

Recovery and portability.

Manifest

Live description of the environment.

25. LEE Mobile Companion

The Android companion remains a first-class interface.

It is not intended to duplicate desktop.

Its purpose is portable awareness and control.

Primary features:

- Today
- Ask Lee
- Capture
- Brief
- Waiting
- Alerts
- Approvals
- Systems
- lightweight controls
- offline capture

26. Mobile Today

The mobile Today screen should quickly answer:

- What matters?
- What changed?
- What am I waiting on?
- Is something blocked?
- Is something at risk?
- Does Lee need a decision?
- Is a connected system unhealthy?

- Is an approval waiting?

27. Mobile Ask Lee

Mobile talks to LEE Core.

It does not independently connect to:

- CIL
- CerbaSeal
- Greyline
- QuantraCore
- Replit AI integrations
- AI providers

The normal pattern is:

Mobile

- LEE Core
- connected systems as required
- result
- Mobile

This keeps authority centralized.

28. Mobile Capture

Mobile should allow rapid capture of:

- text
- screenshots
- photos
- files
- links
- voice notes
- ideas
- observations
- project updates

LEE organizes the information afterward.

29. Mobile Offline Capture

If LEE Core is unreachable, mobile should store captures in an encrypted local queue.

When connectivity returns:

Mobile queue

→ LEE Core

→ Understanding Pipeline

→ Brain

The phone never becomes another authoritative Brain.

30. Mobile Approvals

Mobile should make approvals extremely easy.

Example:

«Greyline requests external export.»

«CerbaSeal verdict: ALLOW.»

«Human confirmation required.»

The app presents:

- proposed action
- target system
- reason
- evidence
- CerbaSeal result
- consequences
- approve
- reject

31. Universal Systems API Layer

All independent Lamont Labs systems should communicate with LEE through standardized APIs.

This applies to:

- CIL
- CerbaSeal
- Greyline
- QuantraCore
- Replit AI Bridge
- future systems

LEE should not import their databases or specialist code.

32. System Registry

LEE should maintain a persistent record for each connected system.

A System Registry entry can include:

- system ID
- name
- version
- description
- system type
- endpoint
- runtime location
- authentication method
- current state
- current health
- last heartbeat
- capabilities
- permissions
- governance requirements
- dependencies
- economics
- owner approval

33. Capability Registry

LEE should reason about capabilities rather than hardcoded project names.

Examples:

Greyline

- production.status
- production.queue
- production.start
- production.pause
- production.resume
- package.validate
- package.build
- package.export
- job.retry

QuantraCore

- system.status
- data.status
- strategy.status
- risk.report
- performance.report
- simulation.run

CIL

- reasoning.resolve
- reasoning.route
- reuse.lookup
- model.select
- health.status

CerbaSeal

- governance.evaluate
- authorization.verify
- policy.version
- health.status

Replit AI Bridge

- ai.execute

- ai.model.status
- ai.usage
- ai.health

34. Standard LEE System Contract

A LEE-compatible project should expose a machine-readable contract.

It should describe:

Identity

What system this is.

Runtime

Where it runs.

Health

How its health is checked.

Capabilities

What LEE may call.

Request Schemas

What each capability accepts.

Response Schemas

What each capability returns.

Events

What it reports back to LEE.

Permissions

What LEE may do.

Risk

How consequential each action is.

Governance

Whether CerbaSeal evaluation is required.

Human Confirmation

Whether owner approval is required.

Economics

Relevant cost information.

Dependencies

What the system relies on.

35. Future Projects Become LEE-Compatible

A future project should not require rewriting LEE.

The integration process should become:

1. Build the project.
2. Expose authenticated API.
3. Implement LEE System Contract.
4. Expose health.
5. Declare capabilities.
6. Declare events.
7. Declare permissions.
8. Declare governance requirements.
9. Connect to LEE.
10. LEE validates the contract.
11. Owner approves.
12. Project becomes available.

36. Connection Modes

Capabilities may be classified as:

Observe

LEE can inspect state and events.

Use

LEE can call the system as a service.

Manage

LEE can operate approved functions.

Governed Manage

LEE may request consequential functions only after the required governance process.

Permissions should exist per capability.

37. Three API Relationships

There are three important API directions.

Service API

LEE → System

LEE uses specialist functionality.

Management API

LEE → System

LEE operates approved workflows.

Event API

System → LEE

The system reports what happened.

This allows both active orchestration and event-driven operation.

38. Greyline Example

LEE might receive:

«Run Greyline overnight and process everything already approved for production. Do not publish externally.»

LEE would:

1. understand the objective
2. inspect Greyline health
3. inspect available capabilities
4. inspect the production queue
5. check permissions
6. consider K6 resource state
7. call Greyline's production API
8. monitor events
9. retry safe failures where permitted
10. stop when human judgment is required
11. summarize the result

Greyline performs the publishing work.

LEE supervises it.

39. QuantraCore Example

QuantraCore retains its own specialist logic.

LEE may observe:

- system health
- data state
- strategies
- risk state
- simulations
- positions

- performance
- runtime problems

LEE may request approved functions such as:

- generate report
- run simulation
- perform health check
- pause approved workflow
- restart a safe worker

QuantraCore's own specialist controls remain inside QuantraCore.

40. System-to-LEE Events

Connected systems should report important events.

Examples:

Greyline

- ProductionStarted
- ProductionCompleted
- PackageReady
- ValidationFailed
- JobHeld

QuantraCore

- DataFeedDegraded
- StrategyPaused
- SimulationCompleted
- RiskConditionDetected

CIL

- ReuseResolved
- ModelSelected
- FrontierEscalated
- ContradictionDetected
- CostAvoided

CerbaSeal

- GovernanceDecisionIssued
- ActionHeld
- ActionRejected
- AuthorizationExpired

Replit AI Bridge

- ModelCallStarted
- ModelCallCompleted
- ModelUnavailable
- IntegrationDegraded

LEE normalizes these into Domain Events.

41. CIL Is Completely External

CIL is a separate API service.

LEE never performs CIL's specialist reasoning-routing logic internally.

LEE sends a cognitive-resolution request to CIL.

CIL determines:

- whether governed reuse is available
- whether new reasoning is required
- what model capability is necessary
- which model or model class should be selected
- whether the request should be held
- how much the chosen route is expected to cost

LEE receives and follows that decision.

42. CIL Is Mandatory Before External Model Calls

For LEE-originated AI reasoning requests:

«CIL must be consulted before LEE sends a model request to Replit or another paid AI route.»

This is a foundational AI-cost-control rule.

The normal path is:

LEE

→ CIL API

CIL attempts:

T1

Governed near-exact reuse.

T2

Governed semantic reuse.

T3

New reasoning/model call required.

Only after T3 does model execution occur.

43. CIL Selects the Model

At T3, CIL does not merely say:

«Use AI.»

CIL determines what model/model class is appropriate.

Factors may include:

- task complexity
- reasoning depth
- modality
- context size
- latency
- confidence requirement
- historical model performance
- cost

- reliability
- task type
- previous success
- current availability
- cost ceiling
- governance constraints

The goal is:

«Use the least expensive valid model route capable of reliably completing the task.»

44. Model Router Has a Narrow Role

LEE may retain a Model Router, but it does not own model-selection intelligence.

CIL

Chooses the cognitive route.

Model Router

Executes the route selected by CIL.

The Model Router handles:

- endpoint selection
- request formatting
- transport
- retries where allowed
- result normalization
- usage capture

It must not silently override CIL's model decision.

45. Correct AI Flow

The long-term path is:

User / Executive Loop / LEE Engine

↓

Identity

↓

Constitution

↓

Intent

↓

Query Engine

↓

Context Economy

↓

CIL API

If CIL returns:

T1 / T2 resolution

LEE receives the result.

No model call occurs.

If CIL returns:

T3 + selected model route

LEE passes that decision to:

Model Router

↓

appropriate execution service

↓

model response

↓

LEE

This ordering is mandatory for normal LEE AI reasoning.

46. Replit AI Bridge

When LEE moves to the K6, a small private Replit-hosted service should remain available.

Its main purpose is:

«Preserve access to Replit-managed AI integrations so desktop LEE does not need to separately obtain and maintain API keys for every underlying AI provider.»

The flow becomes:

LEE Core on K6

→ CIL

→ model selected

→ Model Router

→ Replit AI Bridge

→ Replit-managed AI integration

→ selected model

→ response

→ LEE Core

47. Replit Bridge Is an Execution Service, Not a Decision Maker

The Replit AI Bridge does not decide which model to use.

CIL already made that decision.

If the requested route is unavailable:

Replit AI Bridge

- reports failure/unavailability
- LEE
- CIL
- CIL selects an alternative

The bridge must not silently substitute a more expensive model.

48. Replit Does Not Own the LEE Brain

The Replit bridge should never own:

- Identity
- Constitution
- Event Log
- Fact Ledger
- Interpretation Ledger
- Strategic Anchors
- Institutional Knowledge
- Executive Loop
- Portfolio Intelligence
- Organizational Memory
- authoritative project state

Replit supplies model access and selected cloud capability.

LEE remains locally authoritative.

49. Mobile AI Flow

The Android app needs no model-provider credentials.

Example:

Mobile

- LEE Core on K6
- Identity
- Constitution
- Intent
- Query
- Context

→ CIL API

If CIL resolves:

→ LEE

→ Mobile

If a model is necessary:

→ CIL selects model

→ Model Router

→ Replit AI Bridge

→ Replit-managed model

→ LEE

→ Mobile

The phone does not need:

- OpenAI keys
- Anthropic keys
- Gemini keys
- CIL credentials
- CerbaSeal credentials
- Greyline credentials
- QuantraCore credentials

LEE Core owns those connections.

50. CIL Can Serve Other Lamont Labs Systems Too

CIL should remain independently useful.

Authorized projects may call CIL directly through its API.

For example:

LEE → CIL

Greyline → CIL

Future System → CIL

This allows CIL to become the shared cognitive-routing and AI-cost-control infrastructure for Lamont Labs.

LEE remains the operating orchestrator.

51. CerbaSeal Is Completely External

CerbaSeal is also a separate API service.

LEE never performs CerbaSeal's actual governance decision logic.

LEE prepares a GovernedRequest.

LEE sends it to CerbaSeal.

CerbaSeal returns:

- ALLOW
- HOLD
- REJECT

LEE follows that decision.

There is no local duplicate CerbaSeal engine inside LEE.

52. CerbaSeal Remains Constitutionally Special

CerbaSeal may technically connect through the same Universal Systems API Layer as other systems.

Its authority remains special.

Capabilities can declare:

- consequential = true
- CerbaSeal required = true
- human confirmation = true/false

LEE sees that metadata and knows the required execution path.

53. Consequential-Action Flow

The normal flow becomes:

LEE identifies required capability

→ check permission

→ check risk classification

→ prepare action

→ CerbaSeal API

→ decision

If:

HOLD

Stop.

REJECT

Stop.

CerbaSeal unavailable

Stop.

ALLOW

Continue only if all additional requirements are satisfied.

If owner confirmation is required:

→ Mobile/Desktop approval

Then:

→ target system API

→ result

→ Event Log

54. No Local CerbaSeal Fallback

LEE must never do this:

«CerbaSeal is unavailable, but my local governance logic thinks this is safe.»

That should be architecturally impossible.

If CerbaSeal is required and unavailable:

«HOLD.»

Informational work may continue.

Preparation may continue.

Execution may not.

55. Cross-System Orchestration

LEE should eventually coordinate workflows spanning multiple independent services.

Example:

Greyline reports:

"PackageReady"

LEE:

1. records the event
2. determines validation is required
3. invokes validation capability
4. receives result
5. determines external publication is consequential
6. sends action to CerbaSeal

7. receives ALLOW
8. requests mobile approval
9. receives approval
10. calls Greyline publishing API
11. records the outcome

Every system retains its own specialization.

LEE coordinates the sequence.

56. Health Monitoring

Every connected system should expose standardized health.

Possible states:

- Healthy
- Degraded
- Unreachable
- Failed
- Maintenance
- Unknown

LEE should track:

- last heartbeat
- response latency
- error rate
- consecutive failures
- system version
- dependencies
- workload
- last successful request

This includes:

- CIL
- CerbaSeal
- Replit AI Bridge
- Greyline
- QuantraCore
- future systems

57. Operational Confidence

Connected-system health should contribute to LEE's Operational Confidence.

For example:

LEE may know:

- Brain current
- GitHub current
- Greyline healthy
- CIL healthy
- CerbaSeal unavailable
- several important Facts stale

Operational Confidence might therefore fall even if LEE itself is technically running normally.

This gives the user a measure of how trustworthy LEE's current picture is.

58. Failure Isolation

The system should remain useful when individual components fail.

CIL unavailable

LEE can continue deterministic, memory, retrieval, and non-AI work.

AI reasoning enters degraded state according to policy.

Replit AI Bridge unavailable

LEE remains locally operational.

Replit-hosted model calls cannot execute.

CIL may determine whether another approved route exists.

CerbaSeal unavailable

LEE continues informational work.

Consequential actions requiring CerbaSeal HOLD.

Greyline unavailable

LEE continues operating.

Greyline workflows become unavailable.

QuantraCore unavailable

LEE continues operating.

Internet unavailable

Local desktop LEE remains usable.

Mobile disconnected

Captures queue locally.

Failure of one system should not collapse the environment.

59. Security

System-to-system APIs should support strong authentication.

Depending on deployment, mechanisms may include:

- TLS
- scoped service tokens
- HMAC signing
- timestamps
- nonce/replay protection
- short-lived credentials
- device identity
- network restrictions
- capability-scoped permissions

LEE should never gain unrestricted authority simply because a system is connected.

60. Least Authority

Permissions should exist at the capability level.

For example:

Greyline may permit LEE to:

- inspect queue
- begin approved production
- pause
- retry validation

while:

- publish externally

requires CerbaSeal and human confirmation.

The system should automate safely without collapsing all authority into one master credential.

61. System Economics

LEE should track economics across the environment.

Relevant categories include:

- CIL T1/T2/T3 usage
- model calls
- models selected
- Replit AI usage
- tokens
- AI cost
- frontier calls avoided
- storage
- local compute
- cloud workers
- connected-system costs
- cost by project
- cost by workflow

Metrics should be labeled:

- MEASURED
- ESTIMATED
- UNAVAILABLE

62. CIL Economics

CIL should report metrics such as:

- T1 resolution %
- T2 resolution %
- T3 escalation %
- Frontier Dependency %
- model selection distribution
- average request cost
- latency by route
- estimated avoided model cost
- fallback rate
- performance by task class

LEE should eventually be able to answer:

«How much has CIL saved?»

and:

«What would AI usage have cost without it?»

63. Replit's Long-Term Role

Replit changes roles over time.

Current

Replit hosts most of LEE.

Hybrid

LEE begins moving onto the K6.

K6 Primary

The K6 hosts the authoritative Brain.

Replit remains connected for:

- managed AI integrations
- cloud model execution
- selected cloud workers
- webhooks
- Replit-native capabilities
- remote development capabilities
- possible relay services

The permanent reason to retain the Replit connection is especially:

«AI access without manually maintaining separate provider API keys wherever Replit-managed access is available.»

64. Remote Mobile Access

The Android companion must eventually reach LEE while away from the K6.

The exact networking technology can be chosen later.

Potential mechanisms may include:

- secure relay
- private tunnel
- authenticated cloud broker
- Replit-hosted relay

The architectural requirement is:

«Remote access must not create a second authoritative cloud Brain.»

65. System Manifest

LEE should be able to describe the entire currently connected environment.

The Manifest should include:

- LEE version
- Brain Version
- Identity version
- Constitution
- engines
- capabilities
- connected systems
- system versions
- health
- CIL status
- CIL routing statistics
- CerbaSeal status
- Replit AI Bridge status
- available AI routes
- Semantic Index
- Event Log
- knowledge counts
- backups
- restore verification
- mobile connection
- economics
- Self-Test state

66. Backup and Recovery

LEE should support true recovery.

A real backup should contain the authoritative information required to rebuild the Brain.

Test Restore should:

1. create isolated environment
2. restore backup
3. verify schema
4. rebuild derived state
5. verify invariants
6. compare restored state
7. return PASS/WARN/FAIL

A K6 failure should not mean losing Lee.

67. Time Machine

LEE should be able to reconstruct historical operational state.

Examples:

«What did we believe about this project three months ago?»

«What assumptions were active when this decision was made?»

«What did the system landscape look like before CIL changed?»

Historical state should remain read-only.

68. Simulation

LEE should support structured what-if reasoning.

Examples:

- What happens if CIL is offline for a week?
- What projects are affected if Replit becomes unavailable?
- What happens if Greyline becomes the primary revenue focus?
- What breaks if a shared dependency changes?

Simulations should identify:

- assumptions
- evidence
- uncertainty
- projected outcomes
- affected systems
- confidence

69. Operational Reviews

LEE should convert accumulated history into regular reviews.

Weekly

- movement
- waiting
- blockers
- priorities

Monthly

- patterns
- assumptions
- CIL effectiveness
- cost changes
- project momentum

Quarterly

- portfolio direction
- system value
- resource allocation
- readiness

Annual

- what was believed
- what happened
- which decisions mattered
- what lasting knowledge was earned

70. Operational Self-Improvement

LEE may improve limited operational behavior based on evidence.

Allowed areas may include:

- Brief length
- interruption frequency
- Curiosity thresholds
- Initiative thresholds

- presentation depth
- recommendation timing
- uncertainty calibration

LEE must not autonomously modify:

- Identity
- Constitution
- Facts
- Strategic Anchors
- CerbaSeal authority
- owner permissions

Adaptations must be:

- visible
- logged
- evidence-backed
- reversible

71. Deployment Evolution

Phase 1 — Replit Primary

LEE runs primarily on Replit.

Mobile connects to Replit-hosted LEE.

CIL and CerbaSeal are already separate APIs.

Phase 2 — API Standardization

All specialist systems adopt standardized LEE-compatible API contracts.

Duplicate CIL/CerbaSeal logic is removed from LEE.

Phase 3 — Hybrid K6

Parts of LEE Core move onto the K6.

Replit remains connected.

Mobile continues functioning.

Phase 4 — K6 Primary

The authoritative LEE Brain moves to the K6.

Desktop becomes the main interface.

Android connects remotely.

CIL remains external.

CerbaSeal remains external.

Replit AI Bridge remains connected.

Phase 5 — Mature Distributed Lamont Labs Environment

LEE Core runs locally.

Specialist systems run wherever appropriate.

CIL controls AI routing.

Replit executes selected managed AI calls.

CerbaSeal governs consequential actions.

Mobile provides portable access.

Everything communicates through standardized authenticated APIs.

72. Future AI Flow

The definitive AI path should be:

LEE request

→ Identity

→ Constitution

→ Intent

→ Query Engine

→ Context Economy

→ CIL API

CIL returns either:

REUSE

No model call.

or:

MODEL ROUTE REQUIRED

CIL selects model/model class.

Then:

→ Model Router

→ Replit AI Bridge or another approved execution route

→ selected model

→ result

→ LEE

If the route fails:

→ return failure to CIL

→ CIL makes the fallback decision

The execution layer never silently changes CIL's decision.

73. Future Governance Flow

The definitive consequential-action path should be:

LEE decides an external action is required

→ target capability identified

→ capability contract indicates CerbaSeal required

→ CerbaSeal API

→ ALLOW / HOLD / REJECT

→ human confirmation if required

→ target system API

→ result

→ LEE Event Log

No duplicate local governance decision engine.

74. Final Responsibility Model

LEE

Understands, remembers, prioritizes, coordinates, supervises, and learns.

CIL

Owns cognitive reuse, cost minimization, and model selection.

Model Router

Executes CIL's routing decision.

Replit AI Bridge

Provides Replit-managed AI execution without requiring individual underlying provider API keys on the K6.

CerbaSeal

Owns consequential-action governance.

Greyline

Owns publishing production.

QuantraCore

Owns its specialist quantitative domain.

Future Systems

Own their own specialist domains.

Desktop

Provides full operational control.

Mobile Companion

Provides portable awareness, capture, conversation, alerts, approvals, and lightweight control.

75. Permanent Architectural Invariants

The following rules should be considered foundational:

«CIL and CerbaSeal are independent Lamont Labs systems, not internal LEE engines.»

«LEE communicates with CIL and CerbaSeal only through authenticated APIs.»

«LEE may contain integration clients and contracts but never duplicate their specialist decision logic.»

«CIL is the authoritative cognitive-routing and model-selection system.»

«No normal LEE-originated paid/external model call occurs before CIL evaluates the request.»

«The Model Router executes CIL's decision rather than replacing it.»

«The Replit AI Bridge executes approved model requests but does not independently choose or substitute models.»

«CerbaSeal is the authoritative governance system for capabilities designated as consequential.»

«If CerbaSeal is required and unavailable, execution HOLDS.»

«Independent Lamont Labs systems retain independent code, databases, logic, and safety rules.»

«Future systems connect to LEE through standardized APIs rather than being coded directly into LEE Core.»

«Desktop, mobile, and Replit infrastructure access one authoritative LEE Brain rather than becoming separate Lees.»

76. Final Architecture

USER



LEE DESKTOP — K6

Full operating environment



LEE MOBILE COMPANION — ANDROID

Today

Ask Lee

Capture

Brief

Waiting

Alerts

Approvals

Systems

Lightweight controls



LEE CORE

Identity
Constitution
Intent
Event Log
Memory
Facts
Interpretations
Knowledge
Query Engine
Context Economy
Operational Intelligence
Executive Loop
Portfolio Intelligence
Objectives
Reviews
Institutional Knowledge
System Economics

↓

UNIVERSAL SYSTEMS API LAYER

System Registry
Capability Registry
System Contracts
Authentication
Permissions
Health
Events
Service Calls
Management Calls
Governance Metadata

↓

INDEPENDENT SYSTEMS

CIL
CerbaSeal
Greyline
QuantraCore
Replit AI Bridge
Future Lamont Labs Systems

77. AI PATH

LEE

→ CIL API

CIL determines:

- T1 reuse
- T2 reuse
- new reasoning required
- model/model class
- appropriate route

If model required:

→ Model Router

→ Replit AI Bridge or approved execution route

→ selected model

→ result

→ LEE

78. GOVERNANCE PATH

LEE

→ proposed consequential action

→ CerbaSeal API

→ ALLOW / HOLD / REJECT

→ human confirmation if required

→ target system API

→ result

→ LEE Event Log

79. Final Vision

Project LEE — Learning and Evolving Environment — becomes the persistent operating intelligence surrounding Lamont Labs.

LEE eventually lives primarily on the K6.

The Android companion keeps Lee continuously accessible.

Replit remains connected so desktop LEE can continue using Replit-managed AI integrations without requiring separate underlying AI-provider API keys wherever Replit can provide the capability.

CIL remains completely independent and owns cognitive reuse, cost minimization, and model selection.

CerbaSeal remains completely independent and owns consequential-action governance.

Greyline, QuantraCore, and future Lamont Labs systems remain independent specialists.

All systems communicate through standardized authenticated APIs.

LEE does not contain their expertise.

LEE understands the environment around them.

LEE knows when to use them.

LEE knows when to supervise them.

LEE knows when something deserves attention.

LEE preserves what happened.

LEE learns from the outcome.

And as Lamont Labs grows, new systems can join the environment without turning Project LEE into an increasingly confused monolith.

«Project LEE — Learning and Evolving Environment

One Brain. Desktop and mobile access. Independent specialist systems. Standardized APIs. CIL-controlled AI routing. Replit-managed AI continuity. CerbaSeal-governed execution. Persistent operational intelligence.»